

Question No :8

DS

Session

Tree 1

Question Information

Level 1

Challenge 68

Question description

A company has n employees with certain salaries. Your task is to keep track of the salaries and process queries.

Constraints

- $1 \leq n, q \leq 2 \cdot 10^5$
- $1 \leq p_i \leq 10^9$
- $1 \leq k \leq n$
- $1 \leq x \leq 10^9$
- $1 \leq a \leq b \leq 10^9$

Input

The first input line contains two integers n and q : the number of employees and queries. The employees are numbered $1, 2, \dots, n$.

The next line has n integers $p_1, p_2, \dots, p_n$: each employee's salary.

After this, there are q lines describing the queries. Each line has one of the following forms:

- ! k x : change the salary of employee k to x
- ? a b : count the number of employees whose salary is between $a \dots b$

Output

Print the answer to each ? query.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

5 3
3 7 2 2 5
? 2 3
! 3 6
? 2 3

EXPECTED OUTPUT

3
2

Test Case 2

INPUT (STDIN)

5 3
31 17 12 12 15
? 12 31
! 13 6
? 2 13

EXPECTED OUTPUT

5
3

▼ Mandatory Test Cases

Test Case 1

KEYWORD

int compare(const void
*a,const void *b)

Test Case 2

KEYWORD

void update(int i,int
n,int x)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

1

Test Case 2

TOKEN COUNT

575

Test Case 3

NLOC

65

Code :

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Query {
5      char type;
6      int a;
7      int b;
8  };
9
10 int compare(const void *a, const void *b) {
11     int x = *(const int *)a;
12     int y = *(const int *)b;
13
14     if (x < y)
15         return -1;
16     else if (x > y)
17         return 1;
18
19     return 0;
20 }
21
22 void update(int index, int n, int value, int bit[]) {
23     while (index <= n) {
24         bit[index] += value;
25         index += index & (-index);
26     }
27 }
28
29 int prefixSum(int index, int bit[]) {
```

```
30     int sum = 0;
31
32     while (index > 0) {
33         sum += bit[index];
34         index -= index & (-index);
35     }
36
37     return sum;
38 }
39
40 int lowerBound(int arr[], int n, int value) {
41     int left = 0;
42     int right = n;
43
44     while (left < right) {
45         int mid = left + (right - left) / 2;
46
47         if (arr[mid] < value)
48             left = mid + 1;
49         else
50             right = mid;
51     }
```

```
53     return left;
54 }
55
56 int upperBound(int arr[], int n, int value) {
57     int left = 0;
58     int right = n;
59
60     while (left < right) {
61         int mid = left + (right - left) / 2;
62
63         if (arr[mid] <= value)
64             left = mid + 1;
65         else
66             right = mid;
67     }
68
69     return left;
70 }
71
72 int main() {
73     int n, q;
74
75     scanf("%d %d", &n, &q);
```

```
77     int *salary = malloc((n + 1) * sizeof(int));
78
79     struct Query *queries =
80         malloc(q * sizeof(struct Query));
81
82     /*
83      * Maximum number of different values:
84      * initial n values + q update values
85      */
86     int *values =
87         malloc((n + q + 5) * sizeof(int));
88
89     int valueCount = 0;
90
91     /* Read initial salaries */
92
93     for (int i = 1; i <= n; i++) {
94         scanf("%d", &salary[i]);
95
96         values[valueCount++] = salary[i];
97     }
```

```

97
98
99     /* Read all queries first */
100
101     for (int i = 0; i < q; i++) {
102
103         scanf(" %c %d %d",
104             &queries[i].type,
105             &queries[i].a,
106             &queries[i].b);
107
108         /*
109          For ! k x,
110          k = employee number
111          x = new salary
112          */
113
114         if (queries[i].type == '!') {
115             values[valueCount++] = queries[i].b;
116         }
117     }
118
119     /* Sort all possible salary values */
120
121     qsort(values,
122         valueCount,
123         sizeof(int),

```

```
128     int m = 0;
129
130     for (int i = 0; i < valueCount; i++) {
131
132         if (i == 0 || values[i] != values[i - 1]) {
133             values[m++] = values[i];
134         }
135     }
136
137     /*
138     | Fenwick tree
139     */
140
141     int *bit =
142         calloc(m + 1, sizeof(int));
143
144     /* Add initial salaries */
145
146     for (int i = 1; i <= n; i++) {
147
148         int pos =
149             lowerBound(values, m, salary[i]) + 1;
```

```
150
151     update(pos, m, 1, bit);
152 }
153
154 /* Process queries */
155
156 for (int i = 0; i < q; i++) {
157
158     if (queries[i].type == '!') {
159
160         int employee = queries[i].a;
161         int newSalary = queries[i].b;
162
163         /*
164          | Remove old salary
165          */
166
167         int oldPos =
168             lowerBound(
169                 values,
170                 m,
171                 salary[employee]
172             ) + 1;
```

```
173
174     update(
175         oldPos,
176         m,
177         -1,
178         bit
179     );
180
181     /*
182     | Change salary
183     */
184
185     salary[employee] = newSalary;
186
187     /*
188     | Add new salary
189     */
190
191     int newPos =
192         lowerBound(
193             values,
194             m,
195             newSalary
196         ) + 1;
```

```
197  
198     update(  
199         newPos,  
200         m,  
201         1,  
202         bit  
203     );  
204 }  
205  
206 else if (queries[i].type == '?') {  
207  
208     int a = queries[i].a;  
209     int b = queries[i].b;  
210  
211     /*  
212     |   Number of salaries < a  
213     */  
214  
215     int left =  
216         lowerBound(  
217             values,  
218             m,  
219             a  
220         );  
221  
222     /*  
223     |   Number of salaries <= b
```

```
224     */
225
226     int right =
227         upperBound(
228             values,
229             m,
230             b
231         );
232
233     int answer =
234         prefixSum(right, bit)
235         -
236         prefixSum(left, bit);
237
238     printf("%d\n", answer);
239 }
240
241
242 free(salary);
243 free(queries);
244 free(values);
245 free(bit);
246
247 return 0;
248 }
```

Output :

Test Case :1

	5 3	
	3 7 2 2 5	
	? 2 3	
	! 3 6	
	? 2 3	
	3	
	2	

Test Case :2

	5 3	
	31 17 12 12 15	
	? 12 31	
	! 13 6	
	? 2 13	
	5	
	2	

Keshav Kumar Rauniyar
CH.SC.U4CSE25236